ArrayList VS Linked List 삽입 성능 비교

2022204065 윤성근

목차

과제 설명

코드 작성

결론 도출

과제 설명



ArrayList 와 LinkedList를 활용하여 같은 크기 (10,000)의 초기 리스트 를 준비한 뒤, 임의의 위치에 원소를 x번 삽입하는 데 걸리는 시간과 소요되는 메모리 공간을 측정하고, x를 변화시키며 성능 변화를 관찰한다.

- 매 삽입마다 0 ~ current_size 중 난수로 선택 (random.Random(SEED)

- 재현성을 위해 랜덤 시드를 고정하고 사용할 것

(예시) rng = random.Random(SEED)
rng.randint(0, 10**5)

코드 작성

```python
# run_benchmark.py
from __future__ import annotations
import time, random, tracemalloc
from typing import List, Tuple
from ds import ArrayList, LinkedList  # ds.py와 같은 폴더에 두세요

SEED = 42
INITIAL_SIZE = 10_000
INSERTION_COUNTS = [10, 100, 1000, 5000]

def build_arraylist(n: int) -> ArrayList:
    A = ArrayList()
    for i in range(n):
        A.append(i)
    return A

def build_linkedlist(n: int) -> LinkedList:
    L = LinkedList()
    for i in range(n):
        # ds.LinkedList는 insert(i, v) 혹은 append를 제공
        # 맨 뒤에 붙이는 append가 있다면: L.append(i)
        # 당신 코드엔 append가 있으니 append가 더 빠릅니다:
        L.append(i)
    return L

def gen_indices(num_insertions: int, initial_size: int, seed: int) -> List[int]:
    """매 삽입마다 0~current_size에서 난수를 뽑아 만든 인덱스 시퀀스."""
    rng = random.Random(seed)
    idxs = []
    for k in range(num_insertions):
        current_size = initial_size + k
        idxs.append(rng.randint(0, current_size))
    return idxs

def run_perf_one(ds_type: str, indices: List[int]) -> Tuple[float, int]:
    """주어진 인덱스 시퀀스로 삽입 성능 측정. 시간(s), 피크메모리(KiB) 반환."""
    if ds_type == "ArrayList":
        ds = build_arraylist(INITIAL_SIZE)
    elif ds_type == "LinkedList":
        ds = build_linkedlist(INITIAL_SIZE)
    else:
        raise ValueError("Unknown ds_type")
    tracemalloc.start()
    t0 = time.perf_counter()
    for k, idx in enumerate(indices):
        ds.insert(idx, 99999)  # 임의값
    elapsed = time.perf_counter() - t0
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    peak_kib = peak // 1024
    return elapsed, peak_kib

def main():
    print(f"초기 크기: {INITIAL_SIZE}")
    print("-" * 50)

    for x in INSERTION_COUNTS:
        # ★ 공정성: 같은 인덱스 시퀀스를 두 구조에 적용
        indices = gen_indices(x, INITIAL_SIZE, SEED)

        al_time, al_mem = run_perf_one("ArrayList", indices)
        ll_time, ll_mem = run_perf_one("LinkedList", indices)

        print(f"삽입 횟수 (x): {x}")
        print(f"  ArrayList  - 시간: {al_time:.6f} 초, 피크 메모리: {al_mem} KiB")
        print(f"  LinkedList - 시간: {ll_time:.6f} 초, 피크 메모리: {ll_mem} KiB")
        print("-" * 50)

if __name__ == "__main__":
    main()
```

- 두 구조 모두 동일한 초기 크기(10,000)에서 시작.
- 삽입 횟수 x를 변화시키며, 매 삽입마다 0~current size 범위에서 난수 인덱스 선택.
- SEED = 42로 재현성 확보.
- 공정 비교를 위해 동일 인덱스 시퀀스를 양 구조에 적용.
- 시간은 time.perf_counter, 메모리는 tracemalloc 피크를 사용 .

결론 도출

- 시간 복잡도 면에서
  - 파이썬 리스트(Array List)는 메모리가 연속적이라 랜덤 중간 삽입에도 상대적으로 빠르게 나오는 경우가 많음.
  - LinkedList는 노드 탐색 비용이 매번 듦. 그래서 시간이 더 오래 걸리는 경향을 보임.
- 메모리 면에서
  - LinkedList는 노드 객체 오버헤드 때문에 더 많은 메모리를 사용하게 됨.
  - ArrayList는 재할당 시점에 일시적으로 피크가 생길 수 있지만, 총 오버헤드는 비교적 낮게 유지 됨.
- 정리
  - 랜덤한 중간 위치 삽입을 다수 수행하는 시나리오에서는 실험상 ArrayList가 전반적으로 유리.

```
초기 크기: 10000
--------------------------------------------------
삽입 횟수 (x): 10
  ArrayList  - 시간: 0.000036 초, 피크 메모리: 0 KiB
  LinkedList - 시간: 0.014207 초, 피크 메모리: 1 KiB
--------------------------------------------------
삽입 횟수 (x): 100
  ArrayList  - 시간: 0.000168 초, 피크 메모리: 0 KiB
  LinkedList - 시간: 0.191238 초, 피크 메모리: 8 KiB
--------------------------------------------------
삽입 횟수 (x): 1000
  ArrayList  - 시간: 0.001840 초, 피크 메모리: 93 KiB
  LinkedList - 시간: 1.985411 초, 피크 메모리: 86 KiB
--------------------------------------------------
삽입 횟수 (x): 5000
  ArrayList  - 시간: 0.009684 초, 피크 메모리: 118 KiB
  LinkedList - 시간: 11.879823 초, 피크 메모리: 430 KiB
```

감사합니다